FileVault — Database Schema Design **3**
1. Overview 3
2. Enums 3
2.1 Role 3
2.2 PackageName 3
2.3 FileType 4
2.4 PaymentStatus 4
2.5 ActivityAction 4
3. Tables 5
3.1 users 5
3.2 subscription_packages 7
3.3 user_subscriptions 8
3.4 folders 8
3.5 files 8
3.6 File_versions : 9
3.7 file_shares : 9
3.8 payments 9
3.9 user_sessions : 9
3.10 activity_logs 10
4. Relationships Summary 11
5. Cascade Deletion Rules 12
6. Enforcement Architecture 12

FileVault — Database Schema Design

1. Overview

FileVault is a subscription-based file and folder management SaaS. The database is built on PostgreSQL using Prisma ORM. It consists of 10 tables and 5 enums. The schema enforces subscription-tier restrictions at the data level, tracks full user activity, supports file versioning, public file sharing, Stripe payments, two-factor authentication, and session management.

2. Enums

2.1 Role

Defines the access level of a user account.

Value	Description
ADMIN	Full system access. Can manage packages, view all users, revenue, and analytics.
USER	Standard access. Can manage their own files, folders, and subscription.

2.2 PackageName

The four subscription tiers available in the system. Each maps to exactly one row in `subscription_packages`.

Value	Description
FREE	Default tier assigned at registration. Minimal storage limits.
SILVER	Entry paid tier. Moderate limits.
GOLD	Mid-tier. Higher file size and folder limits.
DIAMOND	Top tier. Maximum limits across all categories.

2.3 FileType

The allowed file categories in the system. Each package defines which of these types a user can upload.

Value	Description
IMAGE	JPG, PNG, GIF, WEBP, and other image formats.
VIDEO	MP4, MOV, AVI, and other video formats.
PDF	PDF documents only.
AUDIO	MP3, WAV, AAC, and other audio formats.

2.4 PaymentStatus

Value	Description
PENDING	Checkout session created but not yet completed.
SUCCEEDED	Payment confirmed by Stripe webhook.
FAILED	Payment attempt was declined or errored.
REFUNDED	Payment was reversed after success.

2.5 ActivityAction

Records every significant user action in the system. Used in the `activity_logs` table.

Value	Description
LOGIN	Successful user login.
LOGOUT	User logged out.
REGISTER	New account created.
LOGIN_FAILED	Failed login attempt (wrong password).
PASSWORD_CHANGED	User changed their own password.
EMAIL_CHANGED	User updated their email address.

TWO_FA_ENABLED	User activated 2FA.
TWO_FA_DISABLED	User deactivated 2FA.
FILE_UPLOAD	A file was uploaded.
FILE_DELETE	A file was moved to trash.
FILE_RESTORE	A file was restored from trash.
FILE_PERMANENT_DELETE	A file was permanently deleted.
FILE_RENAME	A file was renamed.
FILE_MOVE	A file was moved to a different folder.
FILE_DOWNLOAD	A file was downloaded.
FILE_STAR	A file was starred.
FILE_UNSTAR	A file was unstarred.
FOLDER_CREATE	A folder was created.
FOLDER_DELETE	A folder was deleted.
FOLDER_RENAME	A folder was renamed.
TRASH_EMPTIED	All trash items permanently deleted at once.
ACCOUNT_SUSPENDED	Admin suspended a user account.
ACCOUNT_UNSUSPENDED	Admin re-activated a user account.
PASSWORD_RESET_ADMIN	Admin manually reset a user's password.

3. Tables

3.1 users

The central table of the system. Every other table has a direct or indirect relationship with this table. Stores authentication credentials, profile information, account status, and 2FA configuration.

Column	Type	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for the user.
email	VARCHAR	UNIQUE, NOT NULL	User's login email. Must be unique

			across all accounts.
password	VARCHAR	NOT NULL	Bcrypt-hashed password. Never stored in plain text.
firstName	VARCHAR	NOT NULL	User's first name.
lastName	VARCHAR	NOT NULL	User's last name.
role	Role	NOT NULL, default USER	Determines admin or standard access.
isEmailVerified	BOOLEAN	NOT NULL, default false	Set to true after the user clicks the verification link in their email.
emailVerifyToken	VARCHAR	NULLABLE	Temporary token sent to email for verification. Cleared after use.
passwordResetToken	VARCHAR	NULLABLE	Temporary token for password reset. Cleared after use.
passwordResetExpiry	TIMESTAMP	NULLABLE	Expiry time of the password reset token. Reset is rejected after this time.
isSuspended	BOOLEAN	NOT NULL, default false	If true, the user cannot log in. Set by admin.
suspendedAt	TIMESTAMP	NULLABLE	Timestamp when the account was suspended.
suspendedReason	VARCHAR	NULLABLE	Admin-provided reason for suspension.
stripeCustomerId	VARCHAR	NULLABLE	Stripe customer ID linked to this account. Created on first payment.
avatarUrl	VARCHAR	NULLABLE	URL path to the user's uploaded profile picture.
twoFactorSecret	VARCHAR	NULLABLE	TOTP secret key

			used by Google Authenticator. Only set when 2FA is being configured.
twoFactorEnabled	BOOLEAN	NOT NULL, default false	Whether 2FA is fully active for this account.
twoFactorBackupCodes	TEXT[]	NOT NULL, default []	Array of hashed one-time backup codes. Used if the authenticator device is lost.
twoFactorVerified	BOOLEAN	NOT NULL, default false	Whether the user has completed a 2FA-verified login in this session.
createdAt	TIMESTAMP	NOT NULL, default now()	Account creation timestamp.
updatedAt	TIMESTAMP	NOT NULL, auto-update	Last modification timestamp.

Business Rules:

- A suspended user is rejected at the authentication middleware before any token is issued.
- Email verification is required before certain actions are permitted.
- The `twoFactorSecret` is only stored permanently after the user confirms their first OTP code via `confirm2FA`.

3.2 subscription_packages

Defines the four subscription tiers. All limits that control what a user can do are stored here. Admins can update these limits at any time, and changes apply immediately to all users on that package.

Column	Type	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for the package.
name	PackageName	UNIQUE, NOT NULL	The enum tier name (FREE, SILVER, GOLD, DIAMOND).

displayName	VARCHAR	NOT NULL	Human-readable name shown in the UI (e.g. "Gold Plan").
description	TEXT	NULLABLE	Optional marketing description of the package.
maxFolders	INT	NOT NULL	Maximum total folders a user on this plan can create.
maxNestingLevel	INT	NOT NULL	Maximum folder depth (0 = root only, 1 = one sub-level, etc.).
allowedFileTypes	FileType	NOT NULL	Array of permitted file types for upload.
maxFileSizeMB	FLOAT	NOT NULL	Maximum size in MB for a single file upload.
totalFileLimit	INT	NOT NULL	Maximum total number of files across all folders.
filesPerFolder	INT	NOT NULL	Maximum number of files allowed within any single folder.
isActive	BOOLEAN	NOT NULL, default true	If false, the package is hidden from users and cannot be selected.
createdAt	TIMESTAMP	NOT NULL, default now()	When the package was created.
updatedAt	TIMESTAMP	NOT NULL, auto-update	Last modification timestamp.

Business Rules:

- Every upload and folder creation request checks the active package limits before proceeding.
- hen a user downgrades, existing files and folders are not deleted — only new actions are restricted.
- Exactly one row exists per `PackageName` enum value.

3.3 user_subscriptions

Tracks the full subscription history for every user. Each row represents one subscription period. Only one row per user can have `isActive = true` at any time.

Business Rules:

- When a user upgrades or changes plans, the current active row is set to `isActive = false` and `endDate = now()`, and a new row is inserted with `isActive = true`.
- The FREE package is assigned automatically on registration without a Stripe session.
- History of all past packages is preserved and visible to both the user and admin.

3.4 folders

Stores all folders created by users. Supports unlimited nesting via a self-referencing parent relationship, with depth controlled by the user's active package.

Business Rules:

- Before creating a folder, the system checks: (1) total folder count < `maxFolders`, and (2) `nestingLevel` of the new folder ≤ `maxNestingLevel`.
- Deleting a folder cascades to all sub-folders and all files within them.
- `nestingLevel` is calculated as `parent.nestingLevel + 1` at creation time.

3.5 files

Stores metadata for every uploaded file. The actual binary file is stored on disk; this table stores the path and all descriptive metadata.

Column	Type	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for this version record.
fileId	UUID	FK → files.id, CASCADE	The file this version belongs to.
versionNumber	INT	NOT NULL	Sequential version number starting from 1. Increments on each new

			upload.
name	VARCHAR	NOT NULL	Display name at the time this version was saved.
originalName	VARCHAR	NOT NULL	Original filename at the time this version was saved.
mimeType	VARCHAR	NOT NULL	MIME type of this version.
fileType	FileType	NOT NULL	File type category of this version.
size	FLOAT	NOT NULL	File size in bytes of this version.
path	VARCHAR	NOT NULL	Disk path to this version's binary file.
createdAt	TIMESTAMP	NOT NULL, default now()	When this version was saved.

Business Rules:

- Before upload, the system checks: (1) `fileType` is in `allowedFileTypes`, (2) `size` ≤ `maxFileSizeMB × 1024 × 1024`, (3) total file count < `totalFileLimit`, (4) files in this folder < `filesPerFolder`.
- Files with `deletedAt` set are excluded from all normal queries and only visible in the Recycle Bin.
- A cron job runs daily and permanently deletes files where `trashedAt` is older than 30 days.

3.6 File_versions :

Stores previous versions of a file when it is updated. The current version always lives in the `files` table. Old versions are stored here and can be restored at any time.

Business Rules:

- When a new version is uploaded, the current `files` row is snapshotted into `file_versions`, then the `files` row is updated with the new file data.
- Restoring a version copies the version's data back into the `files` row and saves the current state as a new version record.
- Deleting the parent file cascades to all its version records and their disk files.

3.7 file_shares :

Stores public share links for files. Each share has a unique token that can be accessed without authentication. Supports optional expiry and password protection.

Column	Type	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for this share record.
token	VARCHAR	UNIQUE, NOT NULL, default uuid()	The public token embedded in the share URL.
fileId	UUID	FK → files.id, CASCADE	The file being shared.
userId	UUID	FK → users.id, CASCADE	The user who created the share link.
expiresAt	TIMESTAMP	NULLABLE	If set, the link is rejected after this timestamp.
password	VARCHAR	NULLABLE	Bcrypt-hashed password. If set, the accessor must provide it.
accessCount	INT	NOT NULL, default 0	Incremented each time the link is successfully accessed.
isActive	BOOLEAN	NOT NULL, default true	If false, the link is revoked and will return 404.
createdAt	TIMESTAMP	NOT NULL, default now()	When the share was created.
updatedAt	TIMESTAMP	NOT NULL, auto-update	Last modification timestamp.

Business Rules:

- The public access endpoint does not require a JWT token — only the share token.
- If `expiresAt` is in the past or `isActive` is false, the link returns 404.

- `accessCount` is incremented on every successful file access (preview or download).
- A user can have at most one active share per file at a time.

3.8 payments

Column	Type	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for this payment record.
userId	UUID	FK → users.id, CASCADE	The user who made the payment.
packageId	UUID	FK → subscription_packages.id	The user who made the payment.
stripeSessionId	VARCHAR	UNIQUE, NOT NULL	Stripe checkout session ID. Used to prevent duplicate processing.
stripePaymentId	VARCHAR	NULLABLE	Stripe payment intent ID. Set after the webhook confirms success.
amount	FLOAT	NOT NULL	Amount charged in the payment's base currency unit (e.g. dollars).
currency	VARCHAR	NOT NULL, default 'usd'	ISO currency code (e.g. usd, bdt).
status	PaymentStatus	NOT NULL, default PENDING	Payment outcome. Updated by Stripe webhook events.
failureReason	VARCHAR	NULLABLE	Stripe's error message if the payment failed.
createdAt	TIMESTAMP	NOT NULL, default now()	When the checkout session was created.
updatedAt	TIMESTAMP	NOT NULL, auto-update	Last modification timestamp.

Records every Stripe checkout session initiated by a user. Used for the admin revenue dashboard and for sending payment receipt emails.

Business Rules:

- A row is created with `PENDING` status when the Stripe session is created.
- The `checkout.session.completed` webhook updates status to `SUCCEEDED` and activates the subscription.
- The `UNIQUE` constraint on `stripeSessionId` prevents the webhook from processing the same payment twice.

3.9 user_sessions :

Tracks active login sessions per user. Each device or browser that logs in creates one row. Used for the "Active Sessions" feature where users can see and revoke individual sessions.

Column	Type	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for this session.
userId	UUID	FK → users.id, CASCADE	The user this session belongs to.
tokenHash	VARCHAR	UNIQUE, NOT NULL	SHA-256 hash of the refresh token. Used to validate refresh token requests.
deviceInfo	VARCHAR	NULLABLE	User-agent string of the device that created this session.
ipAddress	VARCHAR	NULLABLE	IP address at the time of login.
lastUsedAt	TIMESTAMP	NOT NULL, default now()	Updated each time the refresh token is used to generate a new access token.
createdAt	TIMESTAMP	NOT NULL, default now()	When this session was created (login time).
expiresAt	TIMESTAMP	NOT NULL	When this session expires. Set to `now() + JWT_REFRESH_EXPIRES_IN` at login.

Business Rules:

- The raw refresh token is never stored — only its hash. This prevents token theft from the database.
- Revoking a session deletes its row, making the refresh token permanently invalid.
- Expired sessions are automatically rejected even if the row still exists.

3.10 activity_logs

A full audit trail of every significant action performed in the system. Each row is immutable — never updated, only inserted. Used by users to review their own history and by admins to audit any account.

Column	Column	Constraints	Description
id	UUID	PK, default uuid()	Unique identifier for this log entry.
userId	UUID	FK → users.id, CASCADE	The user who performed the action.
action	ActivityAction	NOT NULL	The type of action performed.
entityType	VARCHAR	NULLABLE	The type of entity affected (e.g. "file", "folder").
entityId	VARCHAR	NULLABLE	The ID of the affected entity.
entityName	VARCHAR	NULLABLE	The name of the affected entity at the time of the action.
metadata	JSON	NULLABLE	Additional context (e.g. old/new name for renames, file size for uploads).
ipAddress	VARCHAR	NULLABLE	IP address from which the action was performed.
userAgent	VARCHAR	NULLABLE	Browser/device string of the client.
createdAt	TIMESTAMP	NOT NULL, default now()	Exact timestamp of the action.

Indexes:

- Composite index on `(userId, createdAt DESC)` for fast paginated queries.

- Index on `action` for filtering by action type.

Business Rules:

- All log writes are fire-and-forget — they never block the main request response.
- Logs older than 90 days are automatically purged by a daily cron job at 3:00 AM.
- Admins can view, filter, and export any user's activity as CSV.

4. Relationships Summary

Relationship	Type	Description
users → user_subscriptions	1 : N	A user has many subscription records over time. Only one is active at any moment.
subscription_packages → user_subscriptions	1 : N	One package can be assigned to many users.
users → folders	1 : N	A user owns all their folders.
folders → folders (self)	1 : N	A folder can contain sub-folders. Depth is controlled by `maxNestingLevel`.
folders → files	1 : N	Every file belongs to exactly one folder.
users → files	1 : N	A user owns all their files. Denormalized FK for fast ownership queries.
files → file_versions	1 : N	A file can have many historical versions. Current version is always in `files`.
files → file_shares	1 : N	A file can have share link records (typically one active at a time).
users → file_shares	1 : N	Tracks which user created each share link.
users → payments	1 : N	A user has a full payment history across all subscription upgrades.
subscription_packages →	1 : N	Records which package was

payments		purchased in each payment.
users → user_sessions	1 : N	Each login device creates a session. Users can revoke individual sessions.
users → activity_logs	1 : N	Every action by a user creates an immutable log entry.

5. Cascade Deletion Rules

All foreign keys use `ON DELETE CASCADE`. This means:

- Deleting a **user** removes all their subscriptions, folders, files, versions, shares, payments, sessions, and activity logs.
- Deleting a **folder** removes all its sub-folders and all files within them (recursive cascade).
- Deleting a **file** removes all its version records and share links.

6. Enforcement Architecture

The subscription enforcement logic works as follows on every protected request:

1. JWT middleware authenticates the user and attaches `userId` to the request.
2. The controller fetches the user's active `user_subscriptions` row joined with `subscription_packages`.
3. The relevant limit is checked against the current database state.
4. If the limit is exceeded, a `403 Forbidden` response is returned with a descriptive message.
5. If all checks pass, the action proceeds and an `activity_logs` row is inserted asynchronously.